



Interactive Data Visualization in R with Shiny

Instructor: **Peter Lawson**

Website: dataservices.library.jhu.edu

Email: dataservices@jhu.edu

JHU Data Archive: archive.data.jhu.edu



Data Services



• About this webinar

- This webinar will be recorded
- This webinar consists of a mix of lectures and hands-on activities
- Teaching Assistants are on hand to help – don't be shy!
- After the workshop, you'll receive a complete version of all R files and workshop materials
- If you have any questions after the workshop, email us at dataservices@jhu.edu

• Using Zoom

- Mute your audio
- Turn off your camera
- To ask questions:
 - Raise your hand – we'll be with you shortly
 - Write in the public chat or private chat a TA directly
 - Unmute your mic and speak up!
- During hands-on activities:
 - Mark **YES** or **NO** to let us know if you're having trouble
 - Write in the public chat



Contact JHU Data Services

GO TO

dataservices.library.jhu.edu

EMAIL

dataservices@jhu.edu

SHARE DATA AT

archive.data.jhu.edu

SURVEY

bit.ly/shiny-r-vis-survey

Helping you



FIND



USE



MANAGE



VISUALIZE



SHARE

DATA



JOHNS HOPKINS
LIBRARIES

Data Services



• This workshop will

1. Introduce basic concepts of interactive data visualization
2. Introduce you to the terminology and syntax for Shiny
3. Walk you through the steps of creating a Shiny application
4. Introduce you to other packages and resources configuring, sharing, and deploying Shiny applications



• What we WON'T cover

Introductory R concepts

- Basic R syntax and terminology
- Installing and loading packages
- R Studio interface

Data cleaning or wrangling in R

Analyzing your data

Introduction to R base graphics and ggplot2

Advanced design and configuration of Shiny apps

- We will only be using the built-in user interface toolkit for designing our Shiny apps. Other toolkits will not be covered, but resources will be provided.



• What we'll cover

1. Introduction to interactive data visualization

2. Introduction to R Shiny

BREAK

3. Designing and Running a Shiny Application

BREAK

4. Wrap Up for Part 1

Data visualization
is the graphical display of
abstract information
for two purposes:

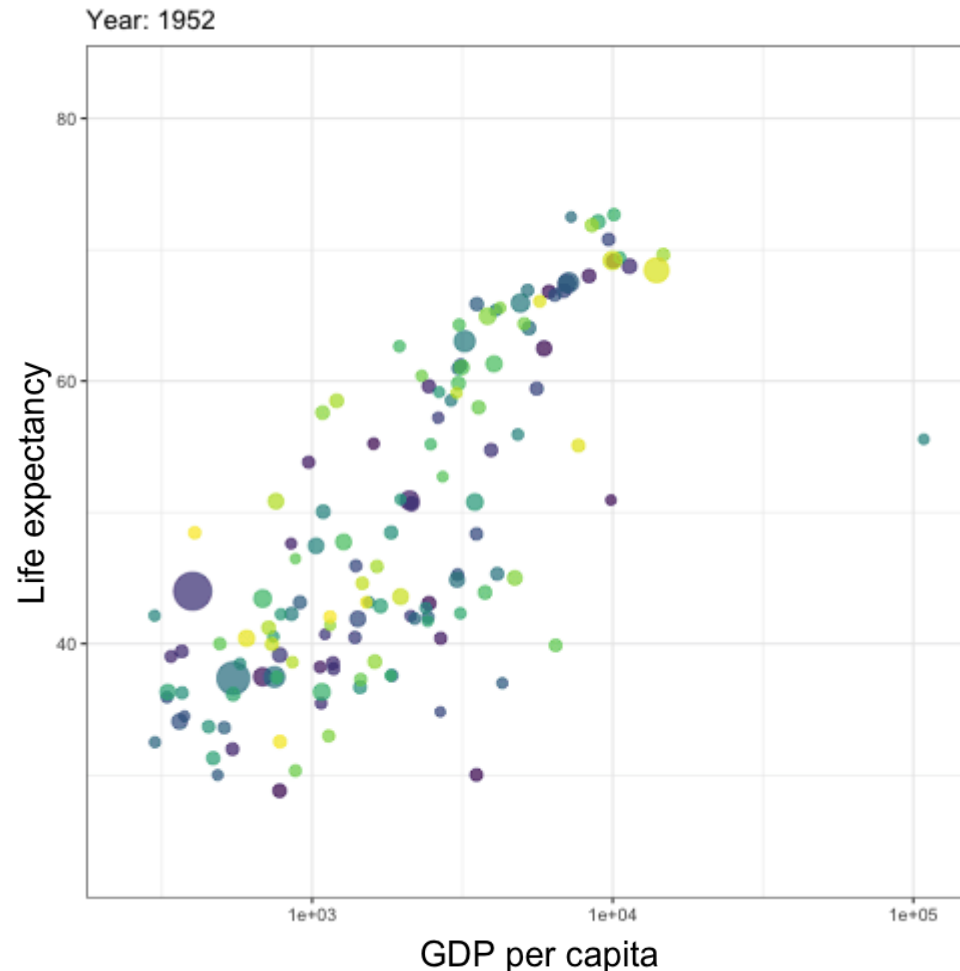
sense-making and communication

- Stephen Few

Data Visualization for Human Perception

Why Dynamic Interactive Visualizations?

- **Telling a Story Through Data**
(Communication)
- **Identifying Patterns and Trends Quickly**
(Sense-Making)
- **Visualizing Complex Multivariate Data**
- **Engaging Your Audience**
- **Collaboration**





What is Shiny?

Shiny is a web application framework for R that allows you to turn your data and visualizations into an interactive web application.

How does Shiny work?

Shiny apps comprise two parts:

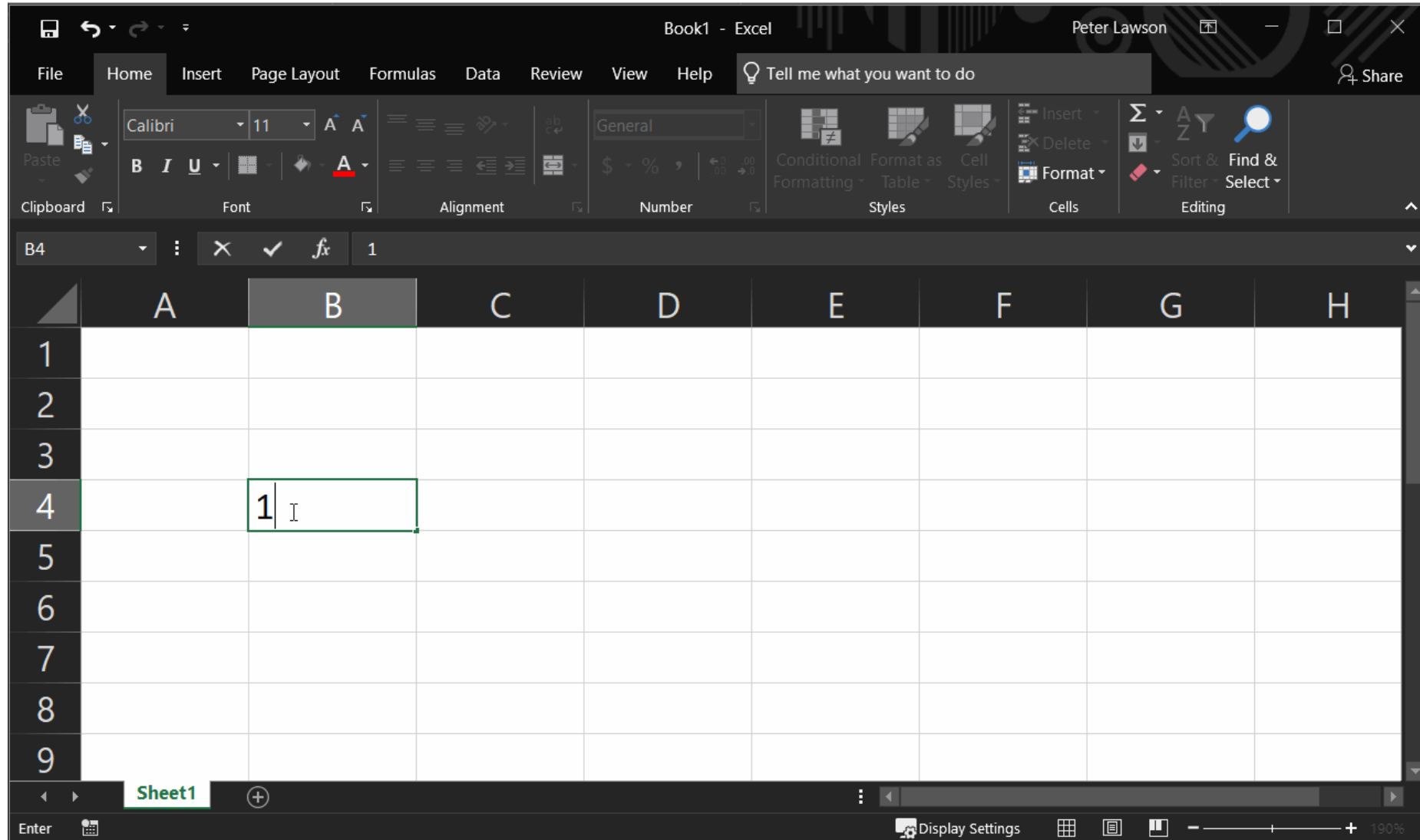
- **User Interface** - Controls the layout and appearance of the script.
- **Server** - Contains the code necessary to build the app.

Shiny uses a concept known as ***Reactive programming***.

Do I need any web development knowledge?

No knowledge of HTML, CSS, or JavaScript is necessary. The look and feel of apps can be improved with HTML and CSS, but this is entirely optional.

What is Reactivity?



What is Reactivity?

The screenshot shows the Microsoft Excel interface with the following details:

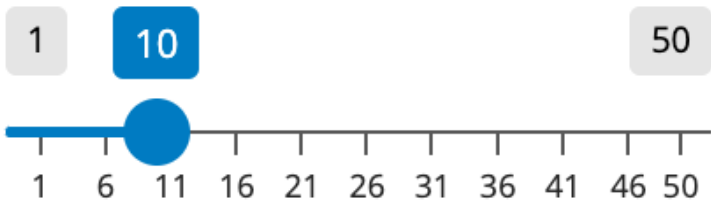
- Title Bar:** Book1 - Excel, Peter Lawson
- Formulas Bar:** E4, $=B4+1$
- Worksheet Grid:**
 - Column headers: A, B, C, D, E, F, G, H
 - Row headers: 1, 2, 3, 4, 5, 6, 7, 8, 9
 - Cell B4 contains the value 1.
 - Cell E4 is highlighted and contains the value 2.
 - A blue arrow points from cell B4 to cell E4.
 - The text "input" is centered over column B, and "output" is centered over column E.
- Bottom Bar:** Sheet1, Ready, Display Settings, 190%

The diagram illustrates reactivity by showing how a change in the input value (cell B4) automatically updates the output value (cell E4) based on the formula $=B4+1$.

What is Reactivity?

input x  output y

Number of bins:



What is Reactivity?

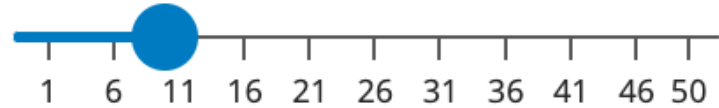
input x  output y

Number of bins:

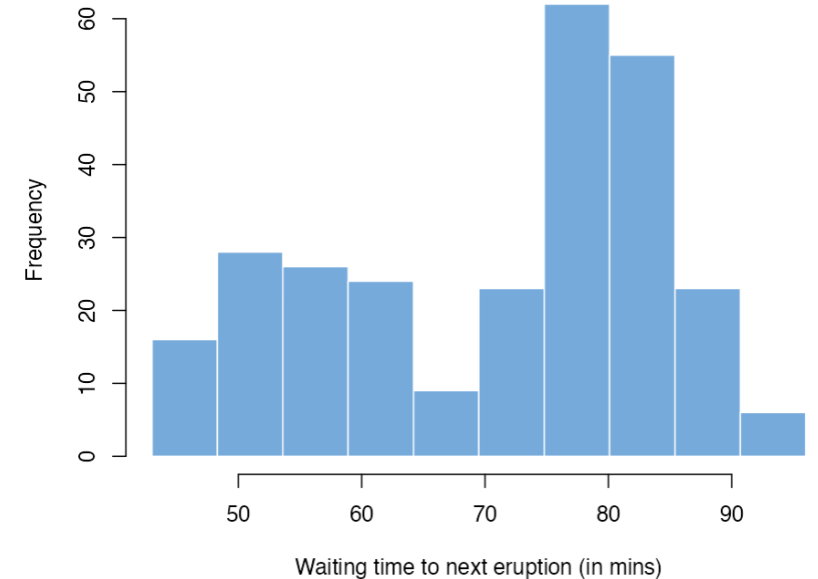
1

10

50



Histogram of waiting times



A Simple R Shiny Application

app.R

```
library(shiny)
library(bslib)

ui <- page_sidebar(
  title = "Hello world!"
)

server <- function(input, output) {

}

shinyApp(ui, server)
```



Modern Web Framework in Shiny

Bootstrap 5

A web framework with HTML, CSS and JavaScript-based design templates for typography, forms, buttons, navigation, and other interface components.



bslib – Bootstrap 5 for Shiny



+



- Allows for creation of Shiny dashboards
- Custom theming of Shiny apps
- Use of modern versions of Bootstrap and Bootswatch (themes for Bootstrap)



Flight Path

Origin

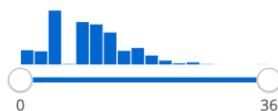
Destination

Carrier

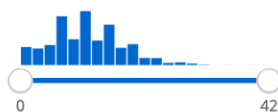
Flight time

Weather

Wind speed



Wind gust



A TOTAL OF

335,177 flights

Across 188 destinations
On 15 different carriers



AVERAGE DEPARTURE

14 mins late

41.5% of flights depart late



AVERAGE ARRIVAL

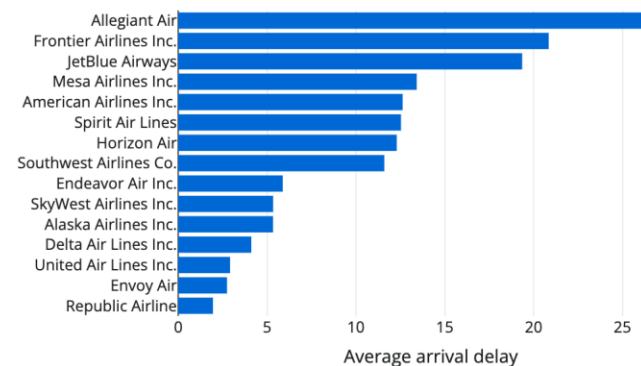
7 mins late

37.7% of flights arrive late

Flight paths



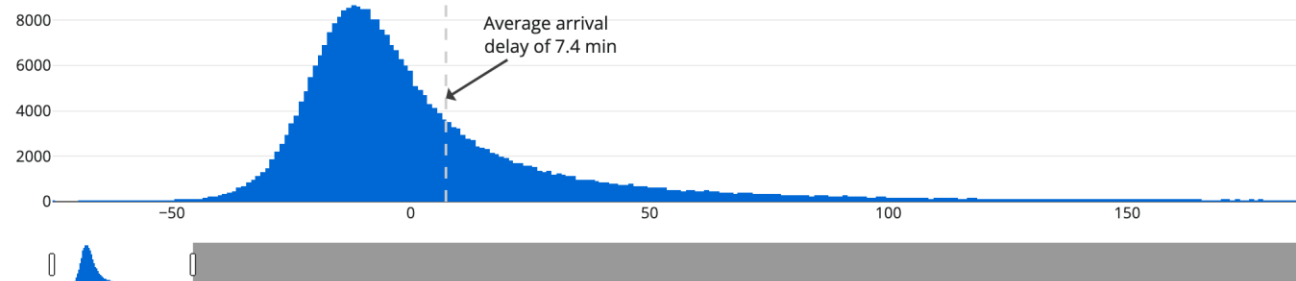
Average delay by category



Distribution of delay times

Overall

Over time



A Simple R Shiny Application

app.R

```
library(shiny)
library(bslib)

ui <- page_sidebar(
  title = "Hello world!"
)

server <- function(input, output) {

}

shinyApp(ui, server)
```



Structure of a Shiny App

ui (User Interface)

```
ui <- page_sidebar(  
  title = "Hello world!"  
)
```

server

```
server <- function(input, output) {  
  
}
```

app.R

```
library(shiny)  
library(bslib)  
  
ui <- page_sidebar(  
  title = "Hello world!"  
)  
  
server <- function(input, output) {  
  
}  
  
shinyApp(ui, server)
```

Structure of a (simple) Shiny App

ui (User Interface)

```
ui <- page_sidebar(  
  title = "Hello world!"  
)
```

The page_sidebar function allows the display to automatically adjust to the browser dimensions.

server

```
server <- function(input, output) {  
  
}
```

app.R

```
library(shiny)  
library(bslib)  
  
ui <- page_sidebar(  
  title = "Hello world!"  
)  
  
server <- function(input, output) {  
  
}  
  
shinyApp(ui, server)
```

Structure of a (simple) Shiny App

ui (User Interface)

```
ui <- page_sidebar(  
  title = "Hello world!"  
)
```

The page_sidebar function allows the display to automatically adjust to the browser dimensions.

server

```
server <- function(input, output) {  
  
}
```

The unnamed function contains the code necessary to run the app.

app.R

```
library(shiny)  
library(bslib)  
  
ui <- page_sidebar(  
  title = "Hello world!"  
)  
  
server <- function(input, output) {  
  
}  
  
shinyApp(ui, server)
```

Structure of a (simple) Shiny App

ui (User Interface)

```
ui <- page_sidebar(  
  title = "Hello world!"  
)
```

The page_sidebar function allows the display to automatically adjust to the browser dimensions.

server

```
server <- function(input, output) {  
  
}
```

The unnamed function contains the code necessary to run the app.

shinyApp constructs and starts a Shiny app from ui and server.

app.R

```
library(shiny)  
library(bslib)  
  
ui <- page_sidebar(  
  title = "Hello world!"  
)  
  
server <- function(input, output) {  
  
}  
  
shinyApp(ui, server)
```


How does a Shiny app Work?

Server



User Interface (UI)

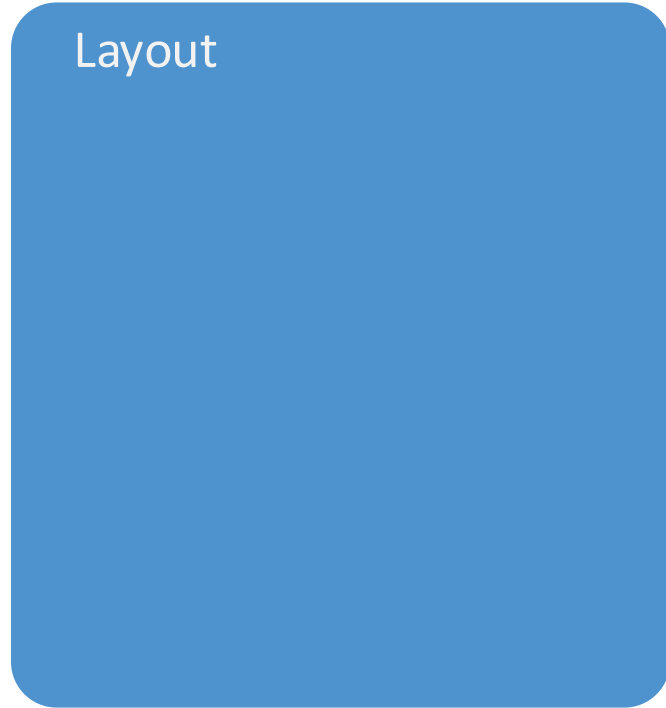
How does a Shiny app Work?

Server



User Interface (UI)

Layout



How does a Shiny app Work?

Server



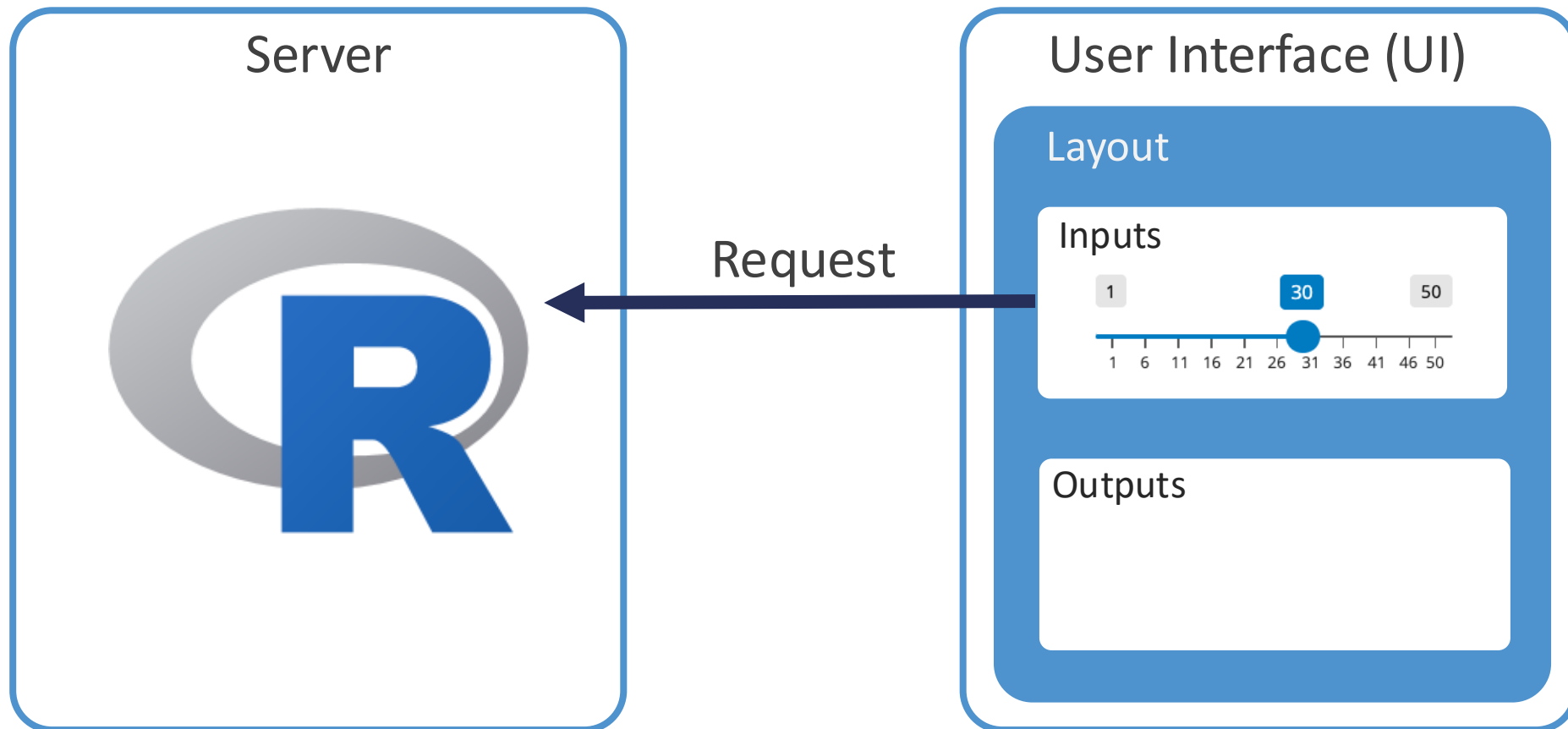
User Interface (UI)

Layout

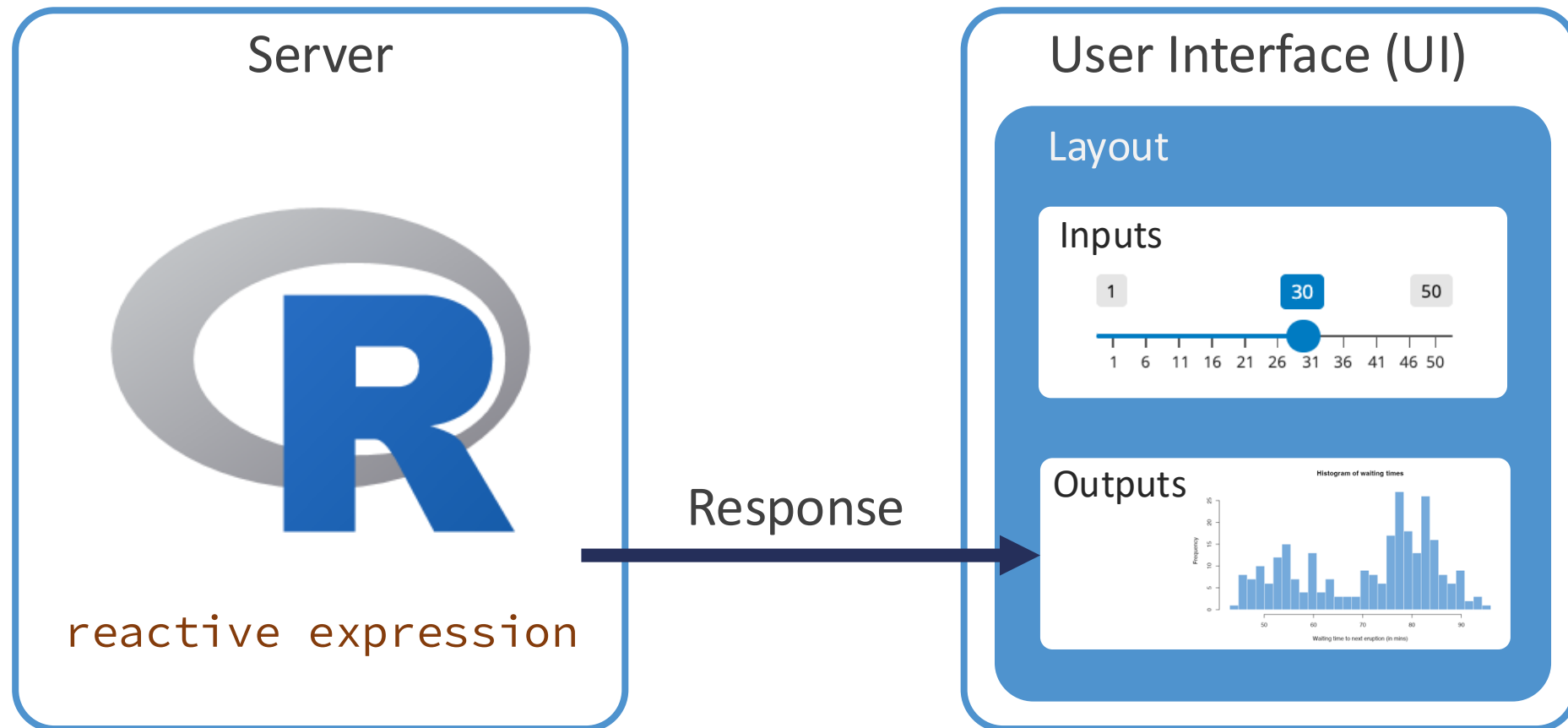
Inputs

Outputs

How does a Shiny app Work?



How does a Shiny app Work?



How does a Shiny app Work?

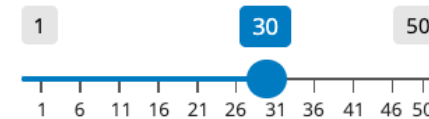
Server



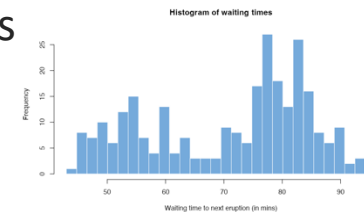
User Interface (UI)

Layout

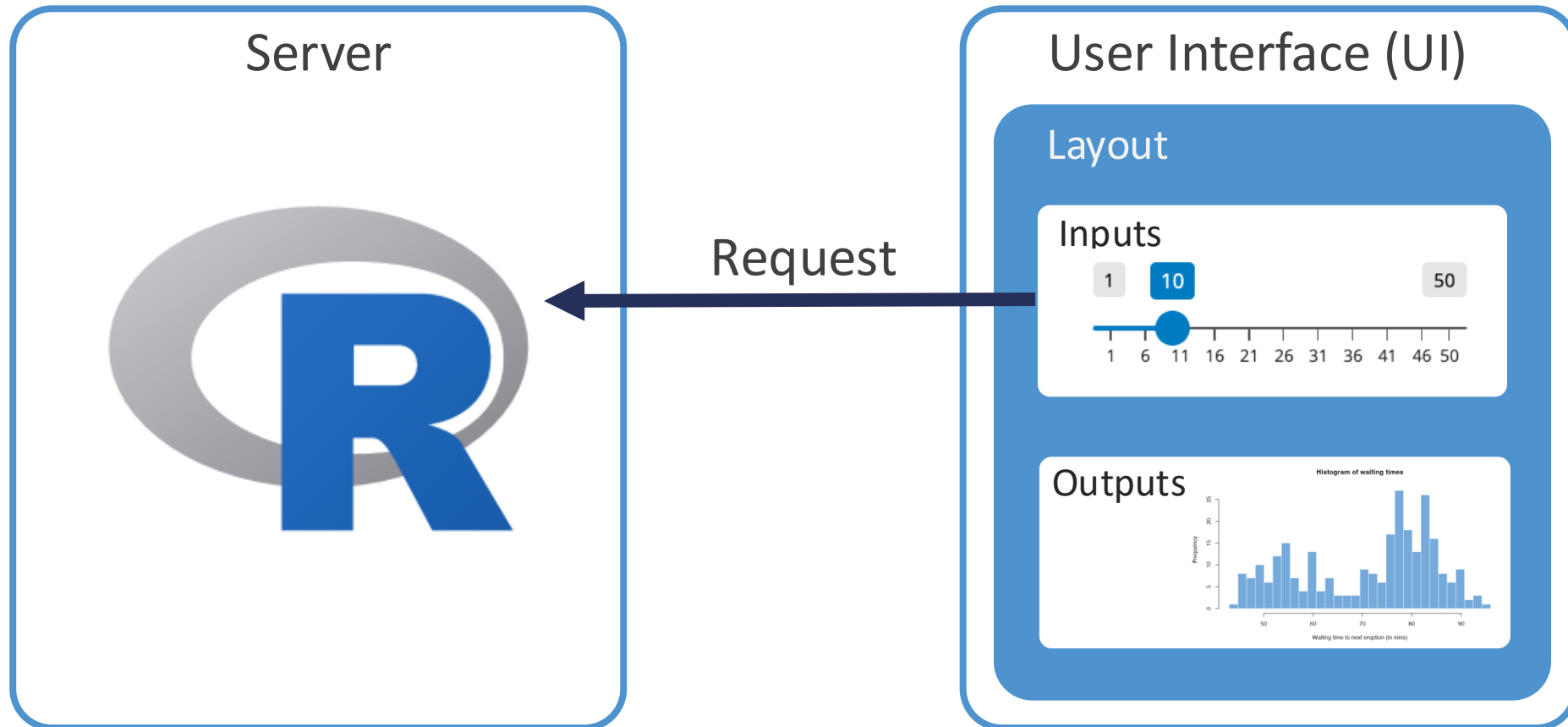
Inputs



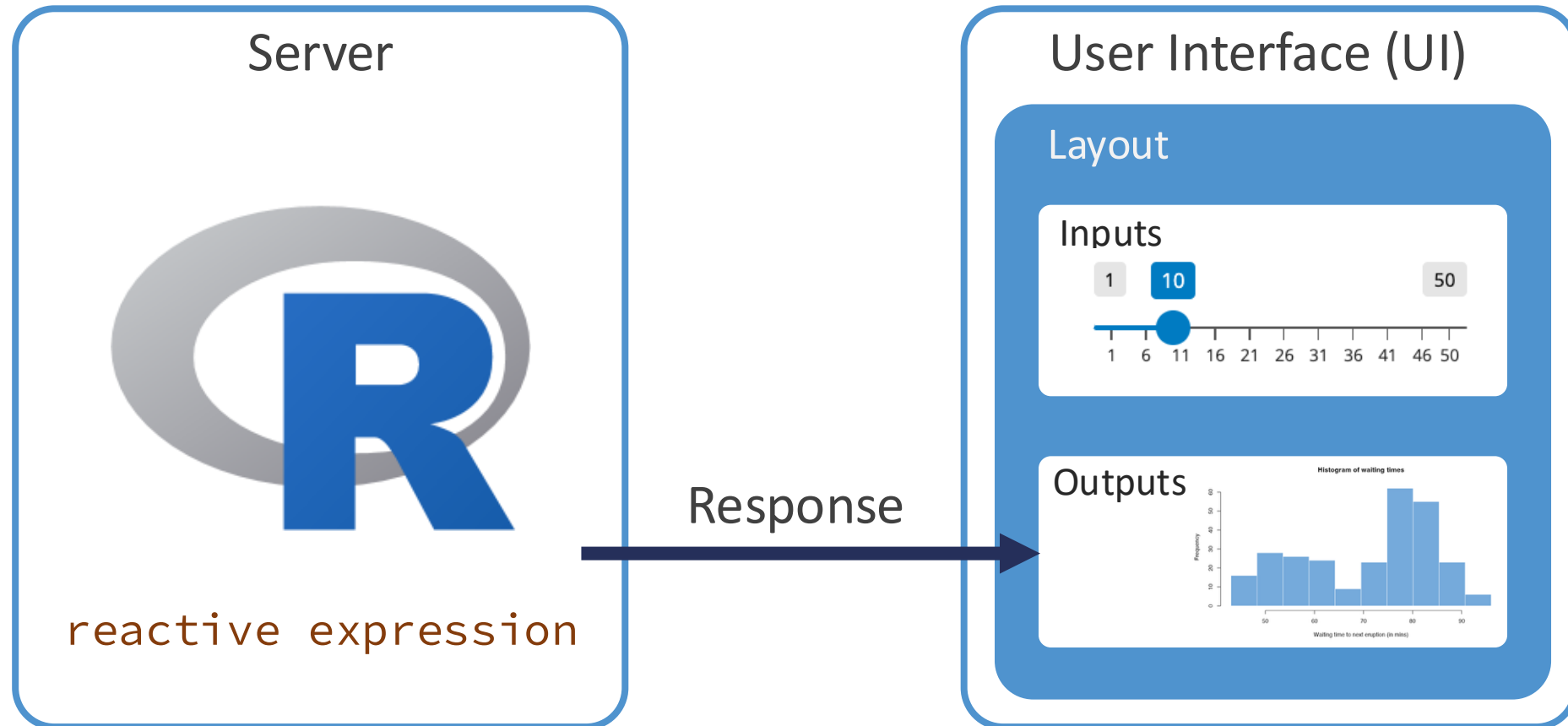
Outputs



How does a Shiny app Work?



How does a Shiny app Work?



Collect user input

Basic widgets

Buttons Action Submit	Single checkbox ☒ Choice A	Checkbox group Select all that apply ☒ Choice 1 ☐ Choice 2 ☐ Choice 3
Date range input Select dates 2024-01-04 to 2024-01-04	File input Browse... No file selected	Help text Note: help text isn't a true widget, but it provides an easy way to add text to accompany other widgets.
Radio buttons Select option ☒ Choice 1 ☐ Choice 2 ☐ Choice 3	Select box Select option Choice 1	Sliders Set value 0 50 100 Set value range 0 25 75 100
Date input Select date 2014-01-01	Numeric input Input number 1	Text input Enter text...

function

widget

`actionButton`

Action Button

`checkboxGroupInput`

A group of check boxes

`checkboxInput`

A single check box

`dateInput`

A calendar to aid date selection

`dateRangeInput`

A pair of calendars for selecting a date range

`fileInput`

A file upload control wizard

`helpText`

Help text that can be added to an input form

`numericInput`

A field to enter numbers

`radioButtons`

A set of radio buttons

`selectInput`

A box with choices to select from

`sliderInput`

A slider bar

`submitButton`

A submit button

`textInput`

A field to enter text

Add reactive expressions

render function	creates
<code>renderDataTable</code>	DataTable
<code>renderImage</code>	images (saved as a link to a source file)
<code>renderPlot</code>	plots
<code>renderPrint</code>	any printed output
<code>renderTable</code>	data frame, matrix, other table like structures
<code>renderText</code>	character strings
<code>renderUI</code>	a Shiny tag object or HTML

Connect reactive output to the UI

Output function	Creates
<code>dataTableOutput</code>	DataTable
<code>htmlOutput</code>	raw HTML
<code>imageOutput</code>	image
<code>plotOutput</code>	plot
<code>tableOutput</code>	table
<code>textOutput</code>	text
<code>uiOutput</code>	raw HTML
<code>verbatimTextOutput</code>	text

Lets Build an App!

Task: Visualize the distribution of waiting times between Old Faithful eruptions.

```
> runExample("01_hello")
```



[Old Faithful at Yellowstone](#) by Mwirig distributed under a CC BY-SA 4.0 License

Adding Reactive Output

Server

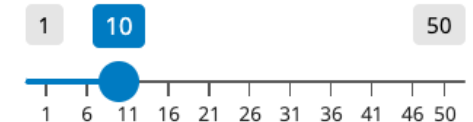
```
server <- function(input, output) {  
  output$distPlot <- renderPlot({  
    x <- faithful$waiting  
    bins <- seq(min(x), max(x), length.out = input$bins + 1)  
  
    hist(x, breaks = bins, col = "#75AADB", border = "white",  
         xlab = "Waiting time to next eruption (in mins)",  
         main = "Histogram of waiting times")  
  })  
}
```

Request

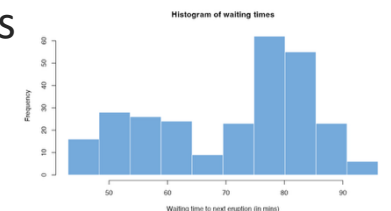
User Interface (UI)

Layout

Inputs



Outputs



Adding Reactive Output

Server

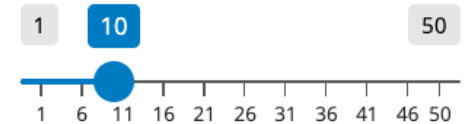
```
server <- function(input, output) {  
  output$distPlot <- renderPlot({  
    x <- faithful$waiting  
    bins <- seq(min(x), max(x), length.out = input$bins + 1)  
  
    hist(x, breaks = bins, col = "#75AADB", border = "white",  
         xlab = "Waiting time to next eruption (in mins)",  
         main = "Histogram of waiting times")  
  })  
}
```

All R input objects that will be used in server need to be defined in the server using the prefix `input$` followed by the `inputID` assigned in the UI, in this case `bins`.

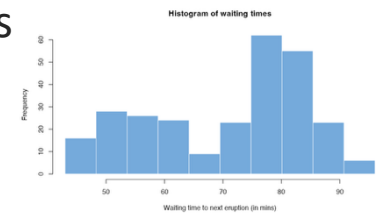
```
sliderInput(inputId = "bins",  
            label = "Number of bins:",  
            min = 1,  
            max = 50,  
            value = 30)
```

Request

Inputs



Outputs



Adding Reactive Output

All R output objects that will be used in the UI need to be defined in the server using the prefix `output$` followed by the name of the object, in this case `distPlot`

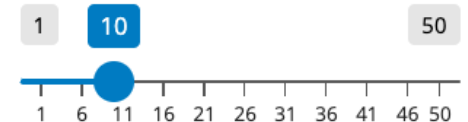
```
server <- function(input, output) {  
  output$distPlot <- renderPlot({  
    x <- faithful$waiting  
    bins <- seq(min(x), max(x), length.out = input$bins + 1)  
  
    hist(x, breaks = bins, col = "#75AADB", border = "white",  
         xlab = "Waiting time to next eruption (in mins)",  
         main = "Histogram of waiting times")  
  })  
}
```

Request

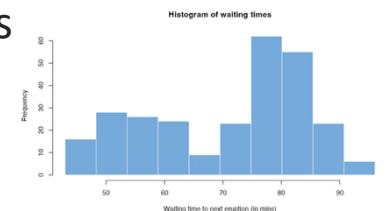
User Interface (UI)

Layout

Inputs



Outputs



Connecting Reactive Output to the UI

Server

```
server <- function(input, output) {  
  output$distPlot <- renderPlot({  
    x <- faithful$waiting  
    bins <- seq(min(x), max(x), length.out = input$bins + 1)  
  
    hist(x, breaks = bins, col = "#75AADB", border = "white",  
         xlab = "Waiting time to next eruption (in mins)",  
         main = "Histogram of waiting times")  
  })  
}
```

Request

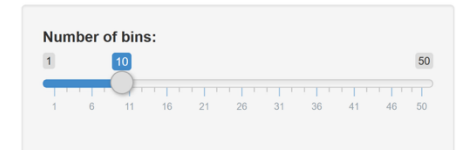
Response



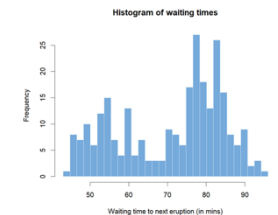
User Interface (UI)

Layout

Inputs



Outputs



Connecting Reactive Output to the UI

Server

```
server <- function(input, output) {  
  output$distPlot <- renderPlot({  
    x <- faithful$waiting  
    bins <- seq(min(x), max(x), length.out = input$bins + 1)  
  
    hist(x, breaks = bins, col = "#75AADB", border = "white",  
         xlab = "Waiting time to next eruption (in mins)",  
         main = "Histogram of waiting times")  
  })  
}
```

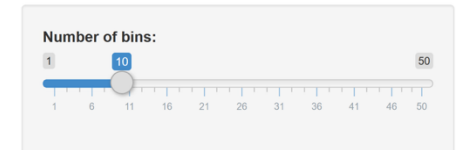
Request

Response

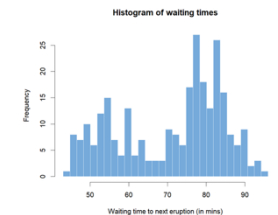
User Interface (UI)

Layout

Inputs



Outputs



```
# Output: Histogram ----  
plotOutput(outputId = "distPlot")
```

Activity

Can you add another plot to the R Shiny app that shows the histogram of eruption times for the `faithful` dataset?

Task:

1. In `server` create a new reactive server function named `output$distPlotEruption` using `renderPlot` to create a histogram of eruption times.

Tips:

- You can copy the code from `output$distPlot` and modify it to display the histogram for `faithful$eruptions`
- Don't change `length.out = input$bins + 1`

2. Within `ui` add a new plot output `plotOutput(outputId = "distPlotEruption")` under the existing plot output

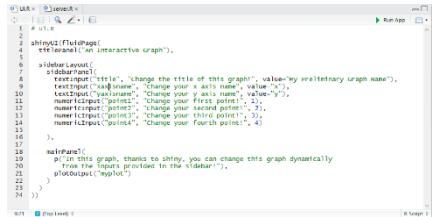
Tips:

- You need to use a comma between objects in the main panel.

View(faithful)

	eruptions	waiting
1	3.600	79
2	1.800	54
3	3.333	74
4	2.283	62
5	4.533	85
6	2.883	55
7	4.700	88
8	3.600	85
9	1.950	51
10	4.350	85
11	1.833	54
12	3.917	84
13	4.200	78
14	1.750	47
15	4.700	83
16	2.167	52
17	1.750	62

Sharing Shiny Apps



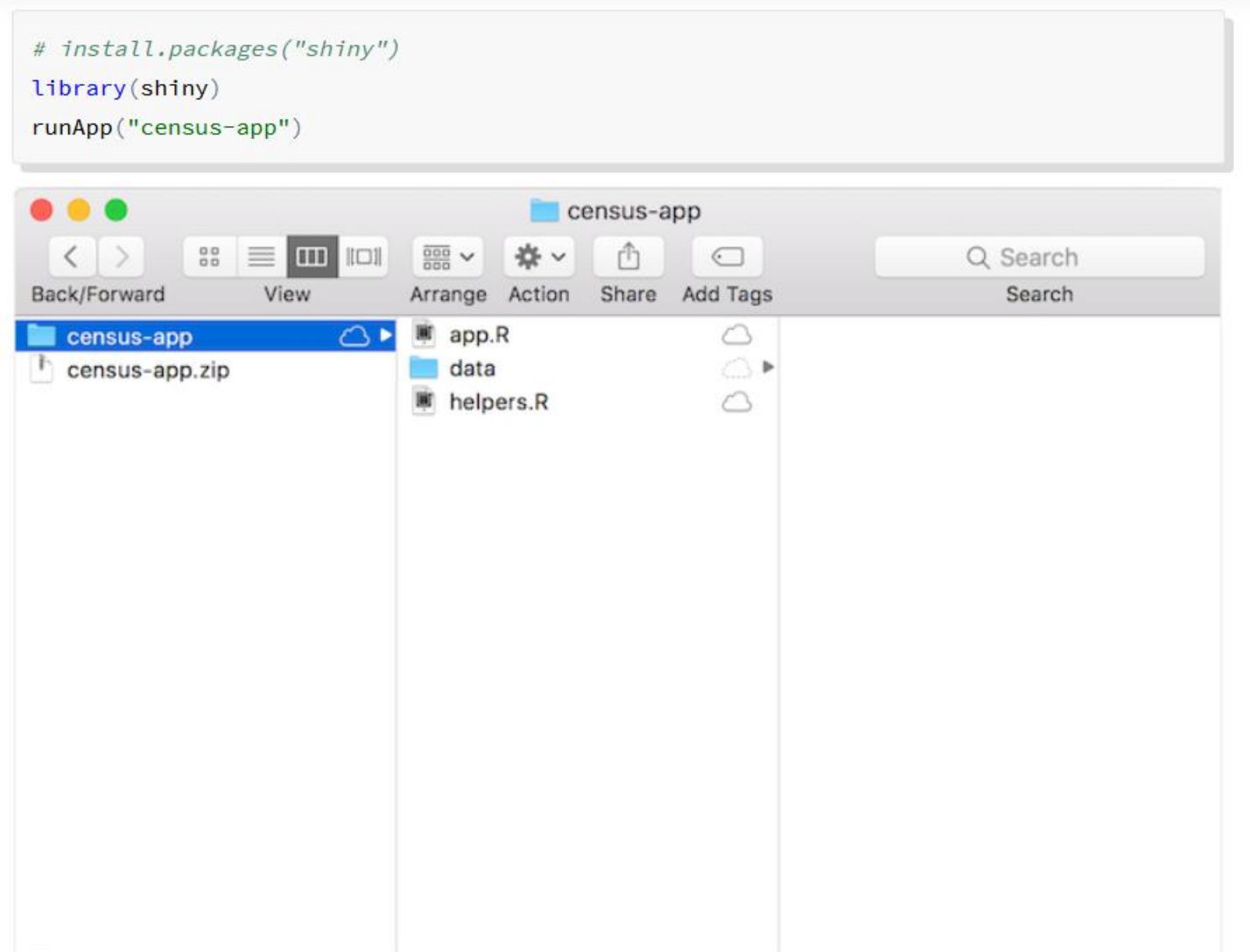
1. Share your Shiny App as an R Script: This is the easiest way to share an R Shiny App, but it requires the user to have R installed on their computers, with the necessary libraries either installed, or readily installable.



2. Share your Shiny app as a web page: This is the most user friendly way of sharing app. Users will be able to access your app on the internet through a web browser. The app can be hosted either through a cloud-service provider, or self-hosted.

Sharing Shiny App as R Scripts

- You will need:
 - app.R
 - Supplementary code and data



Sharing Shiny App as R Scripts

runURL

- Save your Shiny app's directory as a zip file
- Host that zip file at its own link on a web page. Anyone with access to the link can launch the app from inside R by running:

```
library(shiny)  
runURL( "<the weblink>" )
```

Sharing Shiny App as R Scripts

runGitHub

- Store app.R in a GitHub repository, along with any supplemental files the app uses.
- Anyone with access to the Github repository name (if public) can launch the app from inside R by running:

```
runGitHub( "<your repository name>", "<your  
user name>" )
```

Sharing Shiny App as R Scripts

runGist

- GitHub offers a pasteboard service for sharing files at gist.github.com.
- To share your app as a gist:
 - Copy and paste your app.R files to the gist webpage.
 - Record the alphanumeric code returned in your gist URL

```
runGist("eb3470beb1c0252bd0289cbc89bcf36f")
```

Sharing Shiny App as Web Pages

- [Shinyapps.io](https://shinyapps.io): The easiest way to turn your Shiny app into a web page. This is RStudio's hosting service for shiny apps, and it does offer a free account option if your app is hosted publicly.
- [Shiny Server](https://shiny.posit.co/): A free and open-source platform for hosting Shiny apps as web pages using your own Linux server.
- [Posit Connect](https://posit.co/): A publishing platform for the work your teams create in R. Share Shiny applications, R Markdown reports, dashboards, plots, Jupyter Notebooks.
Posit Connect is a licensed commercial software product.

Wrap Up Part 1



• What We Learned

1. Looked at basic concepts of interactive data visualization.
2. Explored what an R Shiny app is, and how it is structured.
3. Learned how to use R Studio Snippets to create a simple Shiny app.
4. Created a more complex Shiny app using reactive programming.
5. How to share your Shiny app!



• What We Will Cover Tomorrow (R-Shiny Part 2)

1. Develop a full-featured interactive dashboard.
2. Using reactive expressions carefully to speed up computation.
3. How to create more complex dashboards and modify visual elements.

Part 2

• Using Zoom

- Mute your audio
- Turn off your camera
- To ask questions:
 - Raise your hand – we'll be with you shortly
 - Write in the public chat or private chat a TA directly
 - Unmute your mic and speak up!
- During hands-on activities:
 - Mark **YES** or **NO** to let us know if you're having trouble
 - Write in the public chat



Contact JHU Data Services

GO TO

dataservices.library.jhu.edu

EMAIL

dataservices@jhu.edu

SHARE DATA AT

archive.data.jhu.edu

SURVEY

bit.ly/shiny-r-vis-survey

Helping you



FIND



USE



MANAGE



VISUALIZE



SHARE

DATA



JOHNS HOPKINS
LIBRARIES

Data Services



• What Covered Yesterday

1. Looked at basic concepts of interactive data visualization
2. Explored what an R Shiny app is, and how it is structured
3. Created a more complex Shiny app using reactive programming
4. How to share your new Shiny app!



• What We Will Cover Today

1. Using external data (palmerpenguins) and packages (ggplot2) in your Shiny app
2. How to create more complex Shiny dashboards and modify visual elements.



• What we'll cover

1. Review of yesterday's materials

2. Replicate the Gapminder Dashboard

BREAK

3. Continue developing our Shiny-based Gapminder Dashboard

BREAK

4. Customizing and Sharing your Shiny Application

5. Wrap Up for Part 2



What is R Shiny?

Shiny is a web application framework for R that allows you to turn your data and visualizations into an interactive web application.

How does Shiny work?

Shiny apps comprise two parts:

- **User Interface** - Controls the layout and appearance of the script.
- **Server** - Contains the code necessary to build the app.

Shiny uses a concept known as ***Reactive programming***.

Do I need any web development knowledge?

No knowledge of HTML, CSS, or JavaScript is necessary. The look and feel of apps can be improved with HTML and CSS, but this is entirely optional.

How Reactive Programming Works

Server

```
server <- function(input, output) {  
  output$distPlot <- renderPlot({  
    x <- faithful$waiting  
    bins <- seq(min(x), max(x), length.out = input$bins + 1)  
    hist(x, breaks = bins, col = "#75AADB", border = "white",  
         xlab = "Waiting time to next eruption (in mins)",  
         main = "Histogram of waiting times")  
  })  
}
```

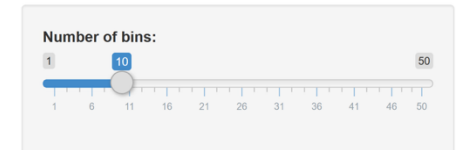
Request

Response

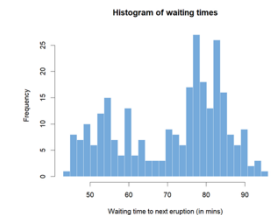
User Interface (UI)

Layout

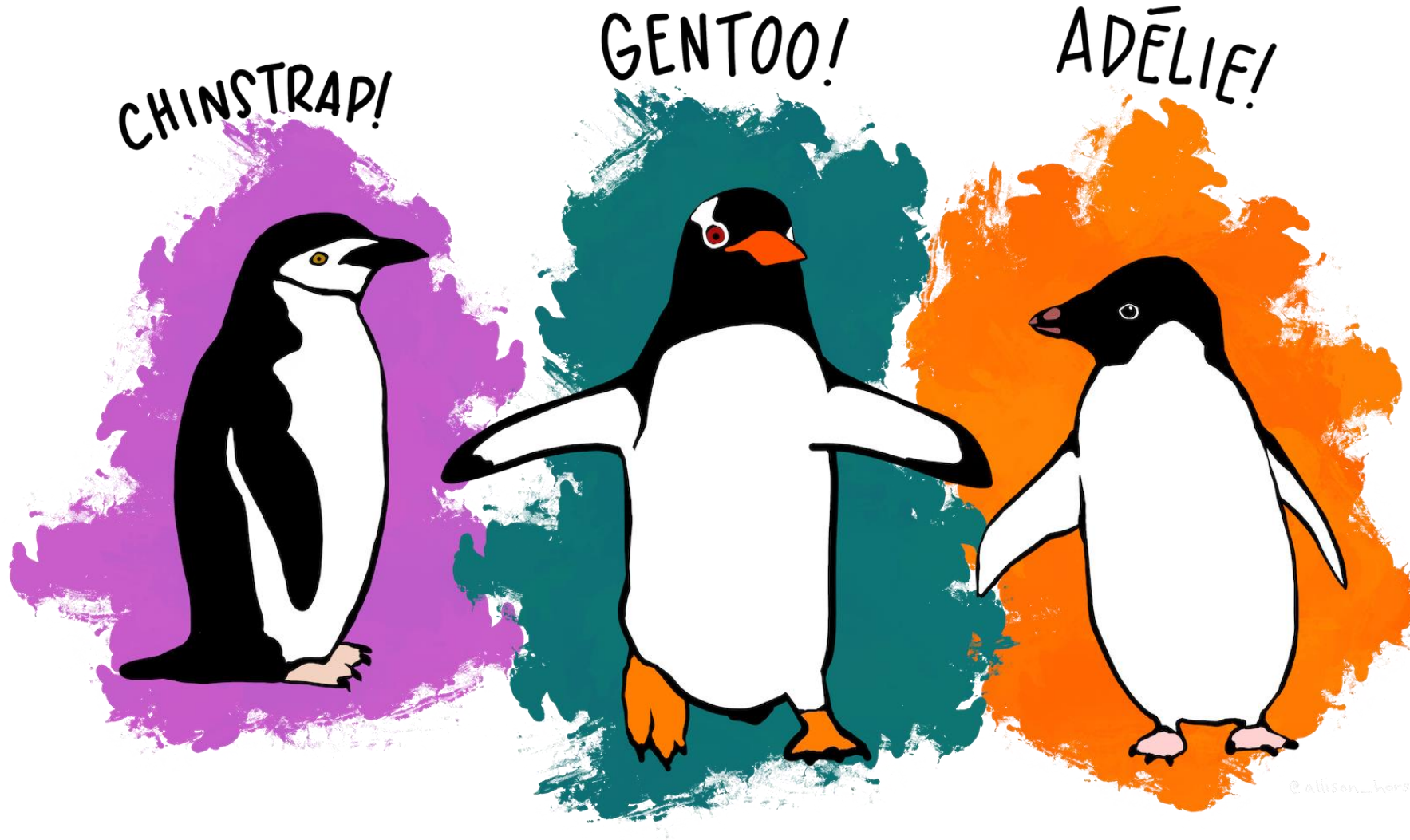
Inputs



Outputs



- Meet the Penguins!





Dr. Kristen Gorman in the field with penguins. Photo: S. Sternbach.

chinstrap



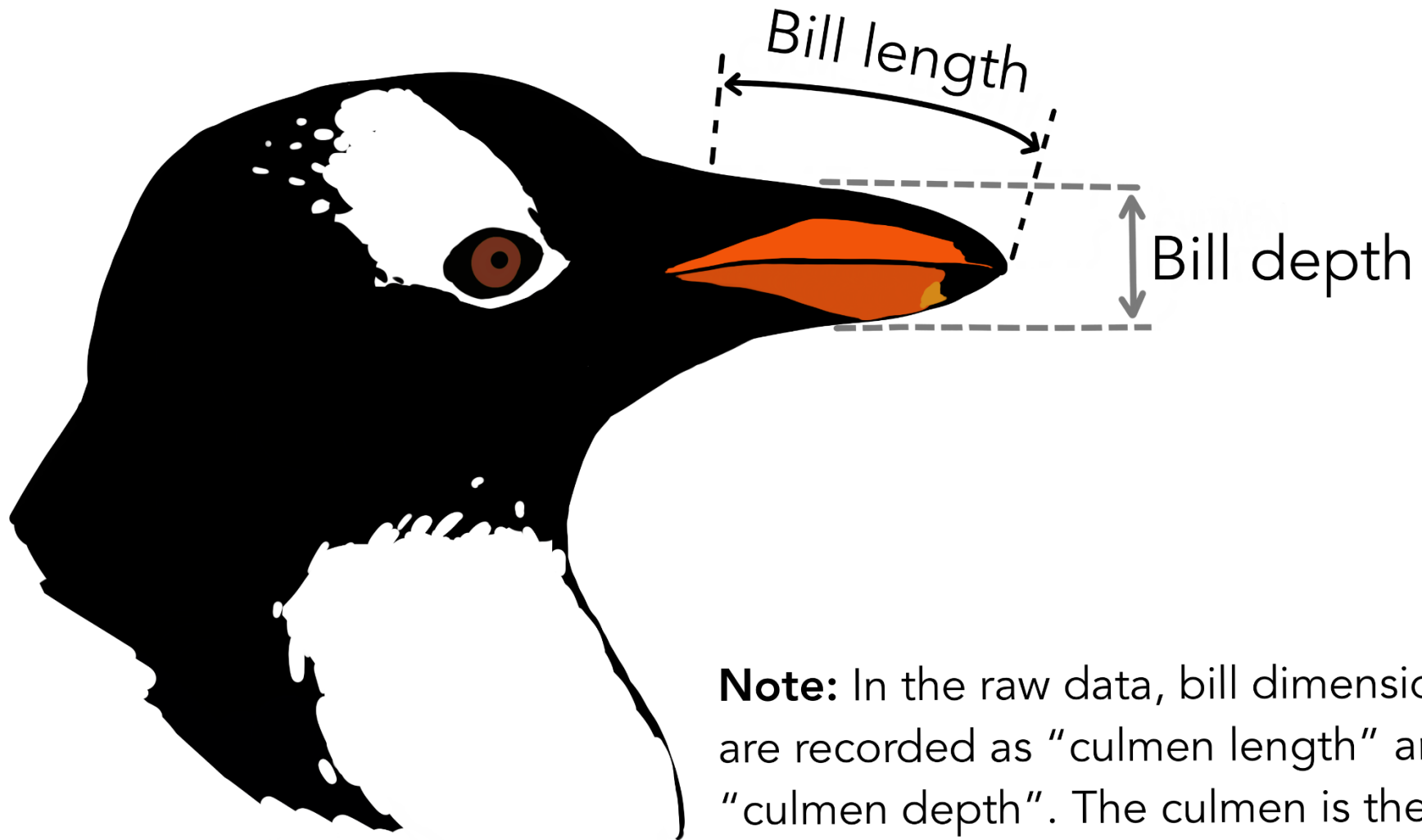
gentoo



Adélie



The three penguin species in the *palmerpenguins* R package. Photos: KB Gorman.



Note: In the raw data, bill dimensions are recorded as "culmen length" and "culmen depth". The culmen is the dorsal ridge atop the bill.

Palmer Penguin Dashboard

X variable

Bill Length (mm)

Y variable

Bill Depth (mm)

Select Penguin Species

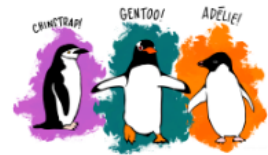
- ☒ Adelie
- ☒ Chinstrap
- ☒ Gentoo

☒ Add smoother

☒ Add facet by island

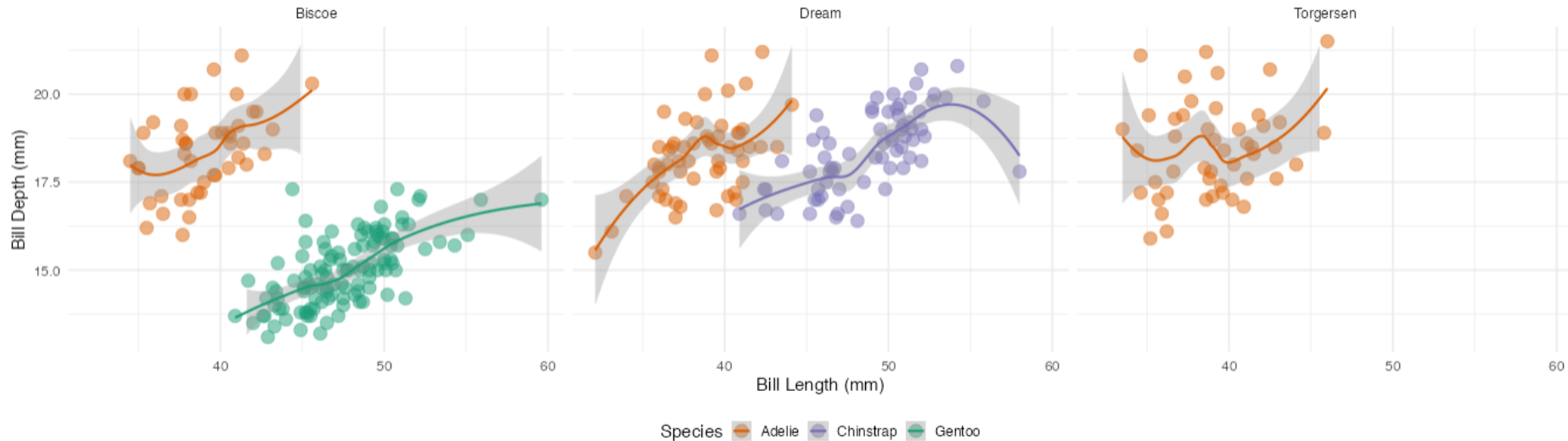
This application uses data from the package:

palmerpenguins



Artwork by @allison_horst

Scatterplot



Summary

Species	Island	Bill Length (mm)	Bill Depth (mm)	Flipper Length (mm)
Adelie	:146	Biscoe	:163	Min. :32.10
Chinstrap	:68	Dream	:123	1st Qu.:39.50
Gentoo	:119	Torgersen	:47	Median :44.50
				Mean :43.99
				3rd Qu.:48.60
				Max. :59.60
Body Mass (g)	Sex	Year		
Min. :2700	female:165	Min. :2007		
1st Qu.:3550	male :168	1st Qu.:2007		
Median :4050		Median :2008		
Mean :4207		Mean :2008		
3rd Qu.:4775		3rd Qu.:2009		
Max. :6300		Max. :2009		

Dataframe

Species	Island	Bill Length (mm)	Bill Depth (mm)	Flipper Length (mm)	Body Mass (g)	Sex	Year
Adelie	Torgersen	39.10	18.70	181	3750	male	2007
Adelie	Torgersen	39.50	17.40	186	3800	female	2007
Adelie	Torgersen	40.30	18.00	195	3250	female	2007
Adelie	Torgersen	36.70	19.30	193	3450	female	2007
Adelie	Torgersen	39.30	20.60	190	3650	male	2007
Adelie	Torgersen	38.90	17.80	181	3625	female	2007
Adelie	Torgersen	39.20	19.60	195	4675	male	2007
Adelie	Torgersen	41.10	17.60	182	3200	female	2007
Adelie	Torgersen	38.60	21.20	181	3800	male	2007

Wrap Up Part 2



• What We Learned Today

1. Using external data (palmerpenguins) and packages (ggplot2) in your Shiny app
2. Using reactive expressions to speed up computation
3. How to create more complex Shiny dashboards and modify visual elements.



THANK YOU!

Take our survey so we can continue to improve and also expand our offerings:

<https://bit.ly/shiny-r-vis-survey>

Interactive Data Visualization in R with Shiny

Instructor: **Peter Lawson**

Part 2

Website: dataservices.library.jhu.edu

Email: dataservices@jhu.edu

JHU Data Archive: archive.data.jhu.edu



Data Services



Resources

- Learn More: <https://shiny.posit.co/r/getstarted/shiny-basics/lesson1/>
- Mastering Shiny Book: <https://mastering-shiny.org/>
- Shiny Cheatsheet:
<https://shiny.posit.co/r/articles/start/cheatsheet/>
- Awesome Shiny Resource List:
<https://github.com/grabear/awesome-rshiny>

Contact JHU Data Services

GO TO

dataservices.library.jhu.edu

EMAIL

dataservices@jhu.edu

SHARE DATA AT

archive.data.jhu.edu

SURVEY

bit.ly/shiny-r-vis-survey

Helping you



FIND



USE



MANAGE



VISUALIZE



SHARE

DATA



JOHNS HOPKINS
LIBRARIES

Data Services